Phase 2 Manual Testing Checklist - Sprint A8

Date: October 27, 2025
Tester: Christian (xian)
Purpose: Systematic validation before Alpha onboarding
Estimated Time: 8-10 hours total

🎯 HOW TO USE THIS CHECKLIST

Testing Approach

1. Run each test in order (they build on each other)
2. Mark status: ✅ PASS, ❌ FAIL, ⚠️ PARTIAL, ⏭️ SKIP
3. For failures: Note what happened, what you expected
4. Take screenshots/logs for evidence
5. Stop if blocker found, report immediately

Status Definitions

- ✅ **PASS**: Works as expected, no issues
- ❌ **FAIL**: Doesn't work, blocks user
- ⚠️ **PARTIAL**: Works but with issues (note them)
- ⏭️ **SKIP**: Can't test (dependency failed)
- 🐛 **BUG**: Issue found, documented separately

Reporting Template

For each failure, create entry:

📋✓

> Test: [Test Number and Name]
> Status: ❌ FAIL
> Expected: [What should happen]
> Actual: [What happened]
> Evidence: [Error message, screenshot, log]
> Severity: BLOCKER / MAJOR / MINOR

📋 TEST SUITE OVERVIEW

Section 1: Setup & Onboarding (30-45 min)

Tests 1-5: First-time user experience

Section 2: Core Conversations (45-60 min)

Tests 6-12: Basic conversational capabilities

Section 3: Preference System (30-45 min)

Tests 13-17: Learning and personalization

Section 4: Document Processing (45-60 min)

Tests 18-24: File upload and analysis

Section 5: GitHub Integration (30-45 min)

Tests 25-30: Issue management

Section 6: Calendar Integration (30-45 min)

Tests 31-36: Schedule management

Section 7: Todo/List Management (30-45 min)

Tests 37-42: Task organization

Section 8: Power User Workflows (60-90 min)

Tests 43-50: Complex multi-tool scenarios

Section 9: Error Handling (45-60 min)

Tests 51-58: Edge cases and failures

🧪 SECTION 1: SETUP & ONBOARDING (30-45 min)

Test 1: Fresh Installation

Objective: Verify first-time setup experience

Steps:

1. Clone repository to new directory
2. Run `python main.py`
3. Follow setup wizard prompts

Expected:

- Setup wizard launches automatically
- Clear instructions for each step
- API key prompts appear
- Preference questionnaire offered
- Success message at end

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 2: API Key Configuration

Objective: Verify API key setup works

Steps:

1. Enter Anthropic API key when prompted
2. Optionally add OpenAI, Google keys
3. Verify keys stored securely

Expected:

- Keys accepted and validated
- Invalid keys rejected with clear error
- Keys stored in encrypted format
- Confirmation message shown

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 3: Preference Questionnaire

Objective: Verify preference collection works

Steps:

1. Complete preference questionnaire
2. Answer personality questions
3. Set working style preferences
4. Save preferences

Expected:

- Questions are clear and relevant
- Answers saved successfully
- Confirmation of saved preferences
- Can review/change preferences later

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 4: First Conversation

Objective: Verify basic chat works

Steps:

1. Start Piper: `python main.py`
2. Send message: "Hello, what can you help me with?"
3. Wait for response

Expected:

- Response within 3 seconds
- Friendly, conversational tone
- Lists capabilities clearly
- No technical errors

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 5: Help Command

Objective: Verify help system works

Steps:

1. Send: "help"
2. Review help content
3. Try: "help github"
4. Try: "help preferences"

Expected:

- General help lists all commands
- Specific help shows relevant details
- Clear, actionable instructions
- Examples provided

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

🧪 SECTION 2: CORE CONVERSATIONS (45-60 min)

Test 6: Simple Query

Objective: Verify basic Q&A works

Message: "What time is it?"

Expected:

- Shows current time
- Shows timezone
- Clean formatting
- Sub-second response

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 7: Identity Query

Objective: Verify self-knowledge

Message: "Who are you?"

Expected:

- Introduces as "Piper Morgan"
- Explains purpose (PM assistant)
- Mentions key capabilities
- Friendly, professional tone

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 8: Status Query

Objective: Verify project status retrieval

Message: "What's my current status?"

Expected:

- Shows project status
- Lists recent activity
- Mentions open items
- Suggests next actions

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 9: Multi-Turn Conversation

Objective: Verify context maintained

Messages:

1. "I'm working on Sprint A8"
2. "What should I prioritize?"
3. "Can you help me with testing?"

Expected:

- Message 2 references Sprint A8
- Message 3 understands testing context
- Coherent conversation flow
- Context maintained across turns

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 10: Compound Intent

Objective: Verify handling of complex messages

Message: "Good morning! Can you check my calendar and suggest what I should work on today?"

Expected:

- Acknowledges greeting
- Checks calendar
- Provides prioritization
- Actionable suggestions

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 11: Clarification Request

Objective: Verify handling of ambiguous input

Message: "Can you help with that thing?"

Expected:

- Asks for clarification
- Suggests what "thing" might be
- Helpful tone, not frustrated
- Guides user to better question

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 12: Empty/Invalid Input

Objective: Verify error handling

Steps:

1. Send empty message (just press Enter)
2. Send only spaces
3. Send very long message (1000+ chars)

Expected:

- Empty: Friendly prompt for input
- Spaces: Same as empty
- Long: Handles gracefully, maybe truncates

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

🧪 SECTION 3: PREFERENCE SYSTEM (30-45 min)

Test 13: State Preference

Objective: Verify preference recording

Message: "I prefer morning meetings because I have more energy then"

Expected:

- Acknowledges preference
- Confirms it's recorded
- No misclassification as time query
- Can retrieve later

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 14: Apply Preference

Objective: Verify preference application

Prerequisites: Test 13 passed

Message: "When should we schedule the architecture review?"

Expected:

- Uses morning preference from Test 13
- Suggests morning time
- Explicitly mentions using preference
- Checks calendar for availability

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 15: Working Style Preference

Objective: Verify style adaptation

Message: "I prefer detailed explanations with examples"

Expected:

- Acknowledges preference
- Subsequent responses more detailed
- Includes examples
- Maintains across session

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 16: View Preferences

Objective: Verify preference retrieval

Message: "What are my current preferences?"

Expected:

- Lists all stored preferences
- Shows morning meeting preference
- Shows working style preferences
- Clean, organized display

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 17: Change Preference

Objective: Verify preference updates

Message: "Actually, I prefer afternoon meetings now"

Expected:

- Acknowledges change
- Updates stored preference
- Confirms update
- New preference applies immediately

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

🧪 SECTION 4: DOCUMENT PROCESSING (45-60 min)

Test 18: Upload Markdown File

Objective: Verify MD file processing

Steps:

1. Create test.md with some content
2. Upload to Piper
3. Ask: "Summarize this document"

Expected:

- File uploaded successfully
- Summary is accurate
- Captures key points
- Proper formatting

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 19: Upload Text File

Objective: Verify TXT file processing

Steps:

1. Create test.txt
2. Upload to Piper
3. Ask: "What's in this file?"

Expected:

- File uploaded successfully
- Content accurately reported
- Can answer questions about it

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 20: Upload Code File

Objective: Verify code file processing

Steps:

1. Create test.py with sample code
2. Upload to Piper
3. Ask: "Review this code"

Expected:

- File uploaded successfully
- Code analysis provided
- Identifies issues/improvements
- Gives specific suggestions

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 21: Upload YAML Config

Objective: Verify config file processing

Steps:

1. Create config.yaml
2. Upload to Piper
3. Ask: "Is this config valid?"

Expected:

- File uploaded successfully
- Validates YAML syntax
- Checks structure
- Suggests improvements

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 22: Multiple Document Context

Objective: Verify handling multiple docs

Steps:

1. Upload 2-3 related documents
2. Ask: "What's the relationship between these docs?"

Expected:

- Understands multiple docs uploaded
- Identifies connections
- Provides synthesis
- Can reference specific docs

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 23: Document Search

Objective: Verify search across documents

Prerequisites: Tests 18-22 passed

Message: "Find mentions of 'testing' in my documents"

Expected:

- Searches all uploaded docs
- Lists matches with context
- Shows document names
- Clickable references

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 24: Large Document Handling

Objective: Verify large file processing

Steps:

1. Upload a large document (>1MB)
2. Ask for summary

Expected:

- Upload completes without timeout
- Processing completes within reasonable time
- Accurate summary despite size
- No memory issues

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

🧪 SECTION 5: GITHUB INTEGRATION (30-45 min)

Test 25: List Issues

Objective: Verify GitHub connection

Message: "List my open GitHub issues"

Expected:

- Connects to GitHub successfully
- Lists open issues
- Shows issue numbers and titles
- Recent issues first

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 26: Create Issue

Objective: Verify issue creation

Message: "Create a GitHub issue: Test issue for Phase 2 validation"

Expected:

- Issue created successfully
- Returns issue number
- Can view on GitHub
- Proper formatting

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 27: Search Issues

Objective: Verify issue search

Message: "Find GitHub issues about testing"

Expected:

- Searches repository
- Returns relevant issues
- Shows issue details
- Can filter by status/labels

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 28: Update Issue

Objective: Verify issue modification

Prerequisites: Test 26 passed

Message: "Add comment to issue #XXX: Testing comment functionality"

Expected:

- Comment added successfully
- Visible on GitHub
- Proper attribution
- Confirmation message

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 29: Close Issue

Objective: Verify issue closure

Prerequisites: Test 26 passed

Message: "Close issue #XXX"

Expected:

- Issue closed successfully
- Status updated on GitHub
- Confirmation message
- Can reopen if needed

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 30: GitHub Error Handling

Objective: Verify graceful failure

Message: "Create issue in non-existent-repo"

Expected:

- Detects invalid repo
- Clear error message
- Suggests correct repos
- Doesn't crash

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

🧪 SECTION 6: CALENDAR INTEGRATION (30-45 min)

Test 31: Check Calendar

Objective: Verify calendar access

Message: "What's on my calendar today?"

Expected:

- Connects to calendar
- Lists today's meetings
- Shows times and titles
- Indicates current/next meeting

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 32: Find Free Time

Objective: Verify availability detection

Message: "When am I free this week?"

Expected:

- Analyzes calendar
- Identifies free slots
- Suggests meeting times
- Respects working hours

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 33: Create Event

Objective: Verify event creation

Message: "Schedule 'Team Sync' tomorrow at 2pm for 30 minutes"

Expected:

- Event created successfully
- Correct time and duration
- Visible on calendar
- Confirmation message

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 34: Calendar with Preferences

Objective: Verify preference application

Prerequisites: Test 13 passed (morning preference)

Message: "When should we schedule our next 1:1?"

Expected:

- Uses morning preference
- Checks calendar availability
- Suggests specific morning times
- Respects existing meetings

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 35: Conflict Detection

Objective: Verify scheduling conflicts handled

Message: "Schedule meeting at [time with existing meeting]"

Expected:

- Detects conflict
- Warns user clearly
- Suggests alternatives
- Doesn't double-book

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 36: Calendar Error Handling

Objective: Verify graceful failure

Steps:

1. Temporarily revoke calendar access
2. Try: "What's on my calendar?"

Expected:

- Detects missing access
- Clear error message
- Instructions to reconnect

- Doesn't crash

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

🧪 SECTION 7: TODO/LIST MANAGEMENT (30-45 min)

Test 37: Create Todo List

Objective: Verify list creation

Message: "Create a todo list for Sprint A8 testing"

Expected:

- List created successfully
- Named appropriately
- Empty initially
- Can add items

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 38: Add Todo Items

Objective: Verify item addition

Message: "Add to Sprint A8 list: Test conversations, Test GitHub, Test calendar"

Expected:

- All three items added
- Correct list
- Proper formatting
- Shows count

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 39: View Todo List

Objective: Verify list retrieval

Message: "Show my Sprint A8 todo list"

Expected:

- Displays all items
- Shows completion status
- Clean formatting
- Items in order added

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 40: Mark Complete

Objective: Verify completion tracking

Message: "Mark 'Test conversations' as complete"

Expected:

- Item marked complete
- Status updated
- Visual indicator (checkbox/strikethrough)
- Confirmation message

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 41: Remove Todo Item

Objective: Verify item deletion

Message: "Remove 'Test GitHub' from my list"

Expected:

- Item removed
- List updated
- Confirmation message
- Remaining items intact

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 42: Todo Persistence

Objective: Verify list persistence

Steps:

1. Create list with items
2. Exit Piper
3. Restart Piper
4. Check list still exists

Expected:

- List persists across sessions
- All items intact
- Completion status preserved
- No data loss

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

🧪 SECTION 8: POWER USER WORKFLOWS (60-90 min)

Test 43: Morning Standup Generation

Objective: Verify standup workflow

Message: "Generate my morning standup"

Expected:

- Reviews recent activity
- Lists what was done yesterday
- Identifies today's priorities
- Notes blockers
- Formatted for easy sharing

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 44: Code Review Assistance

Objective: Verify code review workflow

Steps:

1. Upload code file
2. Ask: "Review this code for best practices"

Expected:

- Thorough code analysis
- Identifies issues
- Suggests improvements
- Explains reasoning
- Actionable feedback

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 45: Documentation Generation

Objective: Verify doc generation workflow

Steps:

1. Upload code file
2. Ask: "Generate API documentation for this"

Expected:

- Creates proper doc structure
- Documents all functions
- Includes examples
- Follows conventions
- Clean formatting

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 46: Multi-Tool Orchestration

Objective: Verify complex workflow

Message: "Check my calendar, find free time this week, create a GitHub issue to schedule architecture review, and add it to my todo list"

Expected:

- Executes all steps
- Proper sequencing
- Each step succeeds
- Final confirmation with details
- All actions visible in respective tools

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 47: Context-Aware Analysis

Objective: Verify deep context usage

Steps:

1. Upload multiple related documents
2. Ask: "What are the inconsistencies across these docs?"

Expected:

- Analyzes all documents
- Identifies conflicts
- Provides specific examples
- Suggests resolutions
- References sources

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 48: Learning from Interaction

Objective: Verify adaptive behavior

Steps:

1. Have extended conversation (10+ turns)
2. State preferences along the way
3. Test if behavior adapts

Expected:

- Remembers conversation context
- Applies stated preferences
- Improves responses over time
- References prior turns naturally

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 49: Sprint Planning Assistance

Objective: Verify planning workflow

Message: "Help me plan Sprint A9. Review A8 results, suggest priorities, estimate effort."

Expected:

- Reviews Sprint A8 data
- Identifies patterns
- Suggests logical next steps
- Provides effort estimates
- Creates actionable plan

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 50: End-of-Day Summary

Objective: Verify daily recap workflow

Message: "Summarize what I accomplished today"

Expected:

- Reviews day's activity
- Lists accomplishments
- Notes incomplete items
- Suggests tomorrow's priorities
- Encouraging tone

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

🧪 SECTION 9: ERROR HANDLING (45-60 min)

Test 51: Network Timeout

Objective: Verify timeout handling

Steps:

1. Disable network temporarily
2. Try GitHub command
3. Re-enable network

Expected:

- Detects network issue
- Clear error message
- Suggests checking connection
- Recovers when network returns
- Doesn't crash

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 52: Invalid API Key

Objective: Verify key validation

Steps:

1. Set invalid Anthropic key
2. Try sending message

Expected:

- Detects invalid key
- Clear error message
- Instructions to fix
- Doesn't expose key
- Doesn't crash

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 53: Permission Denied

Objective: Verify permission handling

Steps:

1. Try accessing resource without permission
2. (e.g., private GitHub repo)

Expected:

- Detects permission issue
- Clear error message
- Suggests granting access
- Doesn't crash

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 54: Rate Limiting

Objective: Verify rate limit handling

Steps:

1. Make rapid repeated requests
2. Trigger rate limit (if possible)

Expected:

- Detects rate limit
- Clear error message
- Suggests waiting
- Doesn't spam API
- Recovers gracefully

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 55: Malformed Input

Objective: Verify input validation

Messages:

1. Send special characters: "!@#$%^&*()"
2. Send SQL injection attempt
3. Send very long string (10,000 chars)

Expected:

- Handles special chars safely
- No SQL injection vulnerability
- Truncates or handles long input
- Doesn't crash or error
- Safe responses

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 56: File Upload Errors

Objective: Verify file handling errors

Steps:

1. Try uploading corrupted file
2. Try uploading unsupported type
3. Try uploading empty file

Expected:

- Detects corruption
- Clear error for unsupported type
- Handles empty file gracefully
- Doesn't crash
- Helpful error messages

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 57: Concurrent Operations

Objective: Verify handling multiple ops

Steps:

1. Start long operation (doc analysis)
2. Try sending another message while processing

Expected:

- Queues or handles gracefully
- Clear status of operations
- Doesn't mix up results
- Completes both successfully

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

Test 58: Recovery from Crash

Objective: Verify crash recovery

Steps:

1. Force crash (Ctrl+C during operation)

2. Restart Piper
3. Check if state recovered

Expected:

- Restarts cleanly
- Session state preserved (or clearly lost)
- No corruption
- Can continue working
- Helpful message about what happened

Actual: _____

Status: [ ] ✅ / [ ] ❌ / [ ] ⚠️ / [ ] ⏭️

Notes: _____

📊 TESTING SUMMARY TEMPLATE

Overall Results

Total Tests: 58
Passed: ___
Failed: ___
Partial: ___
Skipped: ___

Pass Rate: ___%

Critical Blockers Found

1. _____
2. _____
3. _____

Major Issues Found

1. _____
2. _____
3. _____

Minor Issues Found

1. _____
2. _____
3. _____

Positive Findings

1. _____
2. _____
3. _____

Alpha Readiness Assessment

GO / NO-GO: _____

Justification: _____

Conditions for GO: _____

Estimated Fix Time

Blockers: ___ hours
Major Issues: ___ hours
Minor Issues: ___ hours

Total: ___ hours

Revised Alpha Date: _____

🎯 NEXT STEPS AFTER TESTING

1. **Review Results**: Analyze all findings
2. **Prioritize Fixes**: Blockers → Major → Minor
3. **Create Issues**: Document each bug
4. **Fix Critical**: Address blockers first
5. **Re-test**: Verify fixes work
6. **Update Docs**: Document known issues
7. **Reassess**: GO/NO-GO decision
8. **Onboard Alpha**: If GO, proceed with Beatrice

Created: October 27, 2025, 3:15 PM
By: Lead Developer (Sonnet 4.5)
Status: Ready for PM execution
Estimated Time: 8-10 hours total testing